Buổi 6

Authentication và bảo mật giao tiếp API

Ranh giới tin cậy · JWT · refresh rotation · refresh dùng chung · XSS, CSRF, CORS

Mục tiêu buổi học

Sau buổi này, học viên có thể:

1. Vẽ **ranh giới tin cậy** browser/server: client chỉ để trải nghiệm, server mới quyết định quyền
2. Đọc và kiểm tra **JWT**: signature, `exp`, `iss`, `aud`, `roles`; phân biệt **401, 403, 409**
3. Thiết kế **vòng đời phiên**: access token ngắn hạn, refresh token **rotation** và phát hiện **reuse**
4. Viết interceptor **dùng chung một refresh** cho nhiều request 401, không gửi Bearer ra ngoài `apiUrl`
5. So sánh nơi lưu token (bộ nhớ, sessionStorage, cookie `HttpOnly`) cùng rủi ro **XSS, CSRF**; hiểu **CORS** là gì và không là gì

Sản phẩm đầu ra: ma trận kiểm thử bảo mật · test refresh đồng thời · mô tả vòng đời phiên

Lịch buổi học (4 giờ)

Thời gian	Hoạt động
20'	Review buổi 5 (bảng benchmark), phân tích use case
40'	Ranh giới tin cậy, JWT, vòng đời phiên, refresh đồng thời, nơi lưu token, CORS
30'	Live demo: token hết hạn, refresh tranh chấp, token bị lộ
10'	Nghỉ
120'	Lab 5 bài
20'	Review chéo, chốt vòng đời phiên

Use case: "đang làm thì bị đá ra"

Điểm bắt đầu `session06-start`: backend đã có **refresh rotation** (mỗi refresh token dùng một lần), frontend đã có refresh khi gặp 401. Lớp học chạy với `ACCESS_TOKEN_TTL=30s`.

1. `agent1` đăng nhập, mở trang Thuê bao, chờ 45 giây
2. Bấm vào một thuê bao: trang chi tiết gọi **2 request song song**
3. Kết quả: về trang đăng nhập. Tab Network:

Request	Status
`GET /api/subscribers/2000`, `GET /api/tickets?subscriberId=2000`	401, 401
`POST /api/auth/refresh` (lần 1)	**200**
`POST /api/auth/refresh` (lần 2, cùng refresh token)	**401**

> Refresh lần 1 thành công, vậy sao vẫn bị đăng xuất? Cuối buổi sẽ trả lời được.

Ranh giới tin cậy

Trình duyệt (không tin được)

- Người dùng sửa được mọi thứ: JS, sessionStorage, request (DevTools, curl)
- Ẩn nút, guard theo role, kiểm tra form: **trải nghiệm**
- Đọc `roles` trong token để **hiển thị**

Server (quyết định)

- Kiểm tra signature, hạn, người phát, người nhận của token
- `@PreAuthorize("hasRole('SUPERVISOR')")` ở **mọi** API cần quyền
- Kiểm tra dữ liệu: `version` (409), trạng thái, quyền theo bản ghi

> `agent1` không thấy nút "Duyệt", nhưng `curl -X POST /api/tickets/5/approve` với token của agent1 phải trả **403**. Không có test chứng minh thì chưa xong.

401, 403, 409: ba câu hỏi khác nhau

Status	Câu hỏi	Ví dụ Care Portal	Frontend làm gì
401 Unauthorized	Anh là ai? (authentication)	Không token, token hết hạn, sai signature	Refresh một lần; không được thì về đăng nhập
403 Forbidden	Anh được làm việc này không? (authorization)	`agent1` duyệt phiếu, gọi `/api/admin/**`	Báo không có quyền. **Không** refresh, **không** đăng xuất
409 Conflict	Dữ liệu còn như anh thấy không? (nghiệp vụ)	Duyệt phiếu bằng `version` cũ (buổi 2)	Báo dữ liệu đã đổi, cho tải lại

> Refresh khi gặp 403 là lỗi phổ biến: token mới vẫn cùng role, request vẫn 403, có khi lặp vô hạn.

JWT: ký, không mã hóa

```
eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJjYXJlLWFwaSIsImF1ZCI6ImNhcmUtYXBpIi...Ab3x...
└─ header ──────────┘ └─ payload (base64url, AI CŨNG ĐỌC ĐƯỢC) ────┘ └ signature ┘
```

```
{ "iss": "care-api", "aud": "care-api", "sub": "agent1", "token_use": "access",
  "preferred_username": "agent1", "name": "Agent One", "roles": ["AGENT"],
  "iat": 1793328192, "exp": 1793328222 }
```

- Signature HMAC-SHA256 chứng minh **server phát ra và không ai sửa**. Không che giấu gì
- Không đặt secret vào payload (mật khẩu, số CCCD...)
- Frontend chỉ **giải mã** payload để hiển thị tên, role (`jwt.ts`); không kiểm tra signature được (không có key, và không được có)

Server kiểm tra những gì

```
// TokenService.decoder: mọi token đến API đều qua đây
new DelegatingOAuth2TokenValidator<>(
    new JwtTimestampValidator(Duration.ofSeconds(10)), // exp, nbf; clock skew tối đa 10s
    new JwtIssuerValidator("care-api"),                // iss: ai phat
    audience,                                          // aud: phat cho ai
    tokenUse);                                         // access hay refresh
// + macAlgorithm(HS256): cố định thuật toán, không tin "alg" trong header
```

Thử	Kết quả	Vì sao
Sửa `roles` thành `ADMIN`, giữ signature	401	Signature không khớp
`alg: none`	401	Thuật toán cố định HS256
Ký đúng key, hết hạn 15 giây	401	`exp` + clock skew tối đa 10 giây
Refresh token làm Bearer	401	`token_use` sai

Bẫy: mặc định Spring cho clock skew **60 giây**. TTL 30 giây thì token "sống" 90 giây.

Vì sao hai loại token

	Access token	Refresh token
Gửi tới	Mọi API (Bearer)	Chỉ `/api/auth/refresh`, `/api/auth/logout`
Thời hạn	Ngắn: 15 phút (lớp: 30 giây)	Dài: 8 giờ (một ca)
Server lưu trạng thái	Không: không revoke được	Có: bảng `refresh_token`
Lộ thì sao	Dùng được tới khi hết hạn	Dùng được tới khi bị phát hiện → cần **rotation**

> Access token stateless → nhanh, không hỏi DB. Cái giá: đăng xuất xong, access token cũ vẫn dùng được tới khi hết hạn. TTL ngắn là hàng rào.

Refresh rotation và phát hiện reuse

```
login ──► R1 ─refresh─► R2 ─refresh─► R3        (cùng family F, mỗi token dùng MỘT lần)

Kẻ gian lấy được R2 và dùng trước:
  kẻ gian: refresh(R2) ──► R3'   ✓
  app:     refresh(R2) ──► R2 đã dùng ──► revoke CẢ FAMILY F (R3' chết theo) ──► 401
```

```
@Modifying
@Query("""
    update RefreshToken t set t.usedAt = :now
    where t.jti = :jti and t.usedAt is null and t.revokedAt is null and t.expiresAt > :now
    """)
int markUsed(UUID jti, Instant now);   // 2 request cùng lúc: Postgres lock dòng, chỉ 1 câu trả về 1
```

Không "đọc rồi ghi" (race). `@Transactional(noRollbackFor = InvalidRefreshTokenException.class)` : revoke phải được lưu dù sau đó ném 401.

Giờ thì hiểu use case

```
// session06-start: mỗi request 401 tự refresh riêng
return next(withBearer(req, token)).pipe(
  catchError((error) => {
    if (error.status === 401 && token) {
      return auth.refresh().pipe(                    // 2 request -> 2 lần POST /refresh với CÙNG R1
        switchMap((fresh) => next(withBearer(req, fresh))),
        catchError((e) => { auth.login(router.url); return throwError(() => e); }),
      );
    }
    return throwError(() => error);
  }),
);
```

1. Refresh 1 (R1) → 200, nhận R2. Refresh 2 (R1) → **R1 đã dùng** → **revoke family** → 401
2. Nhánh lỗi gọi `auth.login()` → xóa phiên, về đăng nhập

> Backend đúng. Frontend "tự tấn công mình": N request 401 đồng thời → N−1 lần "reuse".

Refresh dùng chung

```
private refreshing: Observable<string> | null = null;

refresh(): Observable<string> {
  this.refreshing ??= this.http
    .post<TokenResponse>(`${this.apiUrl}/api/auth/refresh`, { refreshToken: this.tokens?.refreshToken })
    .pipe(
      tap((tokens) => this.setTokens(tokens)),
      map((tokens) => tokens.accessToken),
      finalize(() => (this.refreshing = null)), // xong -> lần hết hạn sau refresh MỚI
      share(),                                  // mọi người gọi trong lúc chờ dùng chung 1 request
    );
  return this.refreshing;
}
```

Thiếu	Hậu quả (test bắt được)
`share()`	3 request → 3 lần POST
`finalize` reset	Lần hết hạn sau gửi lại **R1 cũ** (body tạo một lần) → bị coi là reuse
Dùng `shareReplay(1)` không reset	Lần sau không refresh, trả token **cũ**

401 về muộn, refresh lỗi

```
const current = auth.accessToken();
if (!current) return throwError(() => error);        // phiên đã kết thúc trong lúc chờ
const fresh$ = current !== token ? of(current)        // request khác vừa refresh xong: dùng luôn
                                 : auth.refresh();
return fresh$.pipe(switchMap((t) => next(withBearer(req, t))));   // gửi lại MỘT lần, không lặp
```

```
// AuthService.refresh: đăng xuất trong PHẦN DÙNG CHUNG -> một lần dù N request đang chờ
tap({ error: (e) => {
  if (e instanceof HttpErrorResponse && e.status === 401) this.login(this.router.url, 'expired');
} })
```

- Refresh **401** (hết hạn, revoke, reuse): đăng xuất một lần, giữ `returnUrl`, `?reason=expired`
- Refresh **mất mạng** (status 0, 5xx): giữ phiên, báo lỗi request, lần sau thử lại
- Đăng xuất chủ động: `POST /api/auth/logout` để server revoke family token

Bearer chỉ cho API của mình

```
// session06-start
if (!req.url.startsWith(apiUrl)) return next(req);   // apiUrl = "https://api.care.vn"
```

URL	`startsWith`	Thực tế
`https://api.care.vn.evil.com/x`	gắn token	host `evil.com` nhận token
`https://api.care.vn@evil.com/x`	gắn token	`api.care.vn` là **username**, host `evil.com`
`https://api.care.vnx/x`	gắn token	domain khác
mọi URL khi `apiUrl = ""` (cùng origin, sau nginx)	gắn token	CDN, bản đồ, analytics đều nhận token

```
const target = new URL(req.url, document.baseURI);
const api = new URL(apiUrl || '/', document.baseURI);
// cùng origin (scheme + host + port) VÀ nằm dưới đường dẫn của apiUrl -> mới là API của mình
```

Nơi lưu token

	Biến JS	`sessionStorage` (hiện tại)	`localStorage`	Cookie `HttpOnly`
XSS lấy được token	Có	Có	Có	**Không**
Còn sau F5	Không	Có	Có	Có
Chia sẻ giữa tab	Không	Không	Có	Có
Trình duyệt tự gửi (CSRF)	Không	Không	Không	**Có**
Gửi sang API khác domain	Dễ	Dễ	Dễ	Khó (CORS credentials, `SameSite=None`)

> Không có lựa chọn "an toàn tuyệt đối". Cookie `HttpOnly` đổi rủi ro XSS lấy rủi ro CSRF. Phương án mạnh nhất cho SPA: **BFF** cùng origin giữ token phía server.

XSS: một lỗi là đủ

- Script chạy trong trang đọc được mọi thứ JS đọc được: `sessionStorage.getItem('care.auth')` → gửi đi, dùng ở máy khác
- Angular **tự escape** khi nội suy `{{ }}` và sanitize `[innerHTML]`. Lỗ thường do chính mình mở:

```
// KHÔNG: ghi chú của khách hàng hiện bằng HTML thô
this.note = this.sanitizer.bypassSecurityTrustHtml(ticket.description);
```

Hàng rào	Tác dụng
Không `bypassSecurityTrust*` với dữ liệu người dùng	Chặn tại nguồn
Content-Security-Policy (`script-src 'self'`)	Script lạ không chạy được (buổi 9, nginx)
TTL access ngắn + phát hiện reuse	Token bị lộ hết giá trị nhanh
Cookie `HttpOnly` cho refresh token	Token không đọc được (nhưng XSS vẫn gọi API ngay trong trang)

Cookie `HttpOnly` thì phải chống CSRF

Trang `evil.com` gửi form `POST https://care.vn/api/tickets/5/approve`: trình duyệt **tự đính cookie** của `care.vn`.

Hàng rào	Ghi chú
`SameSite=Lax` / `Strict`	Cookie không đi theo request chéo site (Lax vẫn cho GET điều hướng)
CSRF token (*double submit*)	Server đặt cookie `XSRF-TOKEN` (JS đọc được); client gửi lại header `X-XSRF-TOKEN`
Không đổi dữ liệu bằng `GET`	`<img src=...>` không làm gì được

```
provideHttpClient(withXsrfConfiguration({ cookieName: 'XSRF-TOKEN', headerName: 'X-XSRF-TOKEN' }))
// Angular CHỈ gắn header cho URL tương đối (cùng origin). API khác domain -> tự làm.
```

Care Portal dùng Bearer trong header (không cookie) nên Spring tắt CSRF (`csrf.disable()`) là hợp lý. Đổi sang cookie thì phải bật lại.

CORS là gì và không là gì

- **Là:** trình duyệt hỏi server "trang ở origin X có được **đọc** response không?" (preflight `OPTIONS`)
- **Không là:** phân quyền. curl, Postman, server khác không gửi `Origin`, không bị CORS

Thử (ma trận #16–18)	Kết quả
Preflight `Origin: http://localhost:4200`	200, `Access-Control-Allow-Origin: http://localhost:4200`
Preflight `Origin: https://evil.example`	403, không có header cho phép
Request thật có Bearer, **không** có `Origin` (curl)	**200**

> Không dùng `allowedOrigins("*")` kèm `allowCredentials(true)` (Spring chặn sẵn). Whitelist đúng origin theo môi trường: `CORS_ORIGINS`.

HTTPS và secret

- **HTTPS ở mọi môi trường thật.** HTTP thì Bearer, cookie, mật khẩu đi dạng rõ trên mạng. HSTS để trình duyệt không bao giờ thử HTTP (buổi 9, nginx)
- **Không có secretKey trong code client.** Mọi thứ trong bundle đều đọc được (DevTools → Sources)
  - Signing key của JWT, client secret, API key trả phí: chỉ ở server, qua biến môi trường
  - `config.json` (runtime config) chỉ chứa thông tin công khai: `apiUrl`, tên môi trường
- Key DEV trong `application.yml` chỉ dùng cho máy học viên. Production truyền `JWT_SECRET` qua secret manager

> Điều kiện bắt buộc của bài cuối khóa: không có secretKey trong repo frontend.

Ma trận kiểm thử bảo mật

Mỗi dòng là một test **chạy lại được**, không phải "đã thử bằng tay một lần".

#	Tình huống	Kỳ vọng	Tầng
3	Sửa `roles` thành `ADMIN`, giữ signature	401	E2E (API)
9	`agent1` gọi API duyệt phiếu	403, phiếu không đổi	E2E (API)
11	`supervisor1` duyệt bằng `version` cũ	409, phiếu không đổi	E2E (API)
13	3 refresh đồng thời cùng token	đúng một 200	E2E (API)
22	3 request cùng 401	đúng một `POST /refresh`	unit
19	Request tới `api@evil.com`	không có `Authorization`	unit

```
npx playwright test -c apps/portal-e2e/playwright.config.mts security --project=chromium
```

Playwright `request`: gọi thẳng API, không cần trình duyệt, không cần Java. Mẫu: `course/_templates/test-matrix.md`.

Live demo (30 phút)

```
ACCESS_TOKEN_TTL=30s docker compose -f infra/docker-compose.yml up -d --build api
```

1. **jwt.io / DevTools:** giải mã token trong sessionStorage. Sửa `roles`, gọi API bằng curl → 401
2. **403 và 409:** curl duyệt phiếu bằng token `agent1` → 403; `supervisor1` với `version` cũ → 409
3. **Tranh chấp refresh:** chờ 45 giây, mở chi tiết thuê bao → bị đá ra. Network: `refresh` 200 rồi 401. Log API: "bi dung lai -> thu hoi ho"
4. **Token bị lộ:** copy refresh token, curl refresh trước app → app bị đăng xuất (và kẻ gian cũng vậy)
5. Chuyển sang `session06-end`, làm lại 3–4: một refresh; bị lộ thì về đăng nhập kèm thông báo "Phiên đăng nhập đã hết hạn"

Lab (120 phút)

Bài	Nội dung	Thời gian
1	Ma trận kiểm thử bảo mật: 401, 403, 409, rotation, CORS bằng Playwright `request`	25'
2	Refresh dùng chung: test đỏ "3 request 401 → 1 refresh", sửa `AuthService.refresh` và interceptor	40'
3	Refresh lỗi và token reuse: đăng xuất một lần, `returnUrl`, `reason=expired`, logout revoke ở server	25'
4	Bearer đúng phạm vi: so origin bằng `URL`, `it.each` URL độc	15'
5	`docs/vong-doi-phien.md`: sơ đồ trình tự, so sánh nơi lưu token, cookie + CSRF	15'

Điểm bắt đầu: `git checkout session06-start` · Đề bài: `course/session-06/lab/README.md`

Bẫy hay gặp khi làm lab

Hiện tượng	Nguyên nhân
Chờ 30 giây mà không thấy 401	Clock skew cho phép 10 giây: chờ ≥ 45 giây; quên `--build api`
Mọi refresh đều 401	Gắn Bearer hết hạn vào chính `/api/auth/refresh` (Spring từ chối cả endpoint `permitAll`)
Test "3 request" vẫn thấy 3 refresh	Mỗi lần gọi `refresh()` tạo observable mới (không giữ `refreshing`), hoặc thiếu `share()`
Refresh lần hai gửi token cũ	Không reset `refreshing` (body tạo một lần lúc dựng observable)
Vòng lặp vô hạn 401 → refresh	Gửi lại qua `HttpClient` (đi lại interceptor) thay vì `next(...)`; refresh cả khi 403
Bị điều hướng về `/login` 3 lần	Gọi `login()` trong `catchError` của từng request thay vì trong phần dùng chung

Tổng kết

1. **Server quyết định quyền**; ẩn nút là trải nghiệm. 401 ≠ 403 ≠ 409
2. JWT được **ký**, không mã hóa. Server kiểm tra signature, thuật toán cố định, `exp` (và clock skew), `iss`, `aud`, loại token
3. Access ngắn + refresh **rotation**, dùng lại thì **revoke cả family**
4. Frontend: **một** refresh dùng chung, gửi lại một lần, đăng xuất một lần, xử lý 401 về muộn
5. Bearer chỉ cho **cùng origin** với `apiUrl`, không so chuỗi đầu
6. Không có nơi lưu token an toàn tuyệt đối: XSS ↔ CSRF. CORS không phải phân quyền. HTTPS, không secretKey trong client

Buổi 7: SSO với Keycloak (OIDC, PKCE) và chiến lược kiểm thử: hành trình Agent tạo → Manager duyệt.